NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 8

TOPIC : WRITE THE ASSEMBLY CODES FROM PAGES 18-21 SHOWN IN CHAP6_YTHA_YU SLIDE

COURSE CODE : CSE 0611-3322

COURSE TITLE : MICROPROCESSOR & ASSEMBLY LANGUAGE SESSIONAL.

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

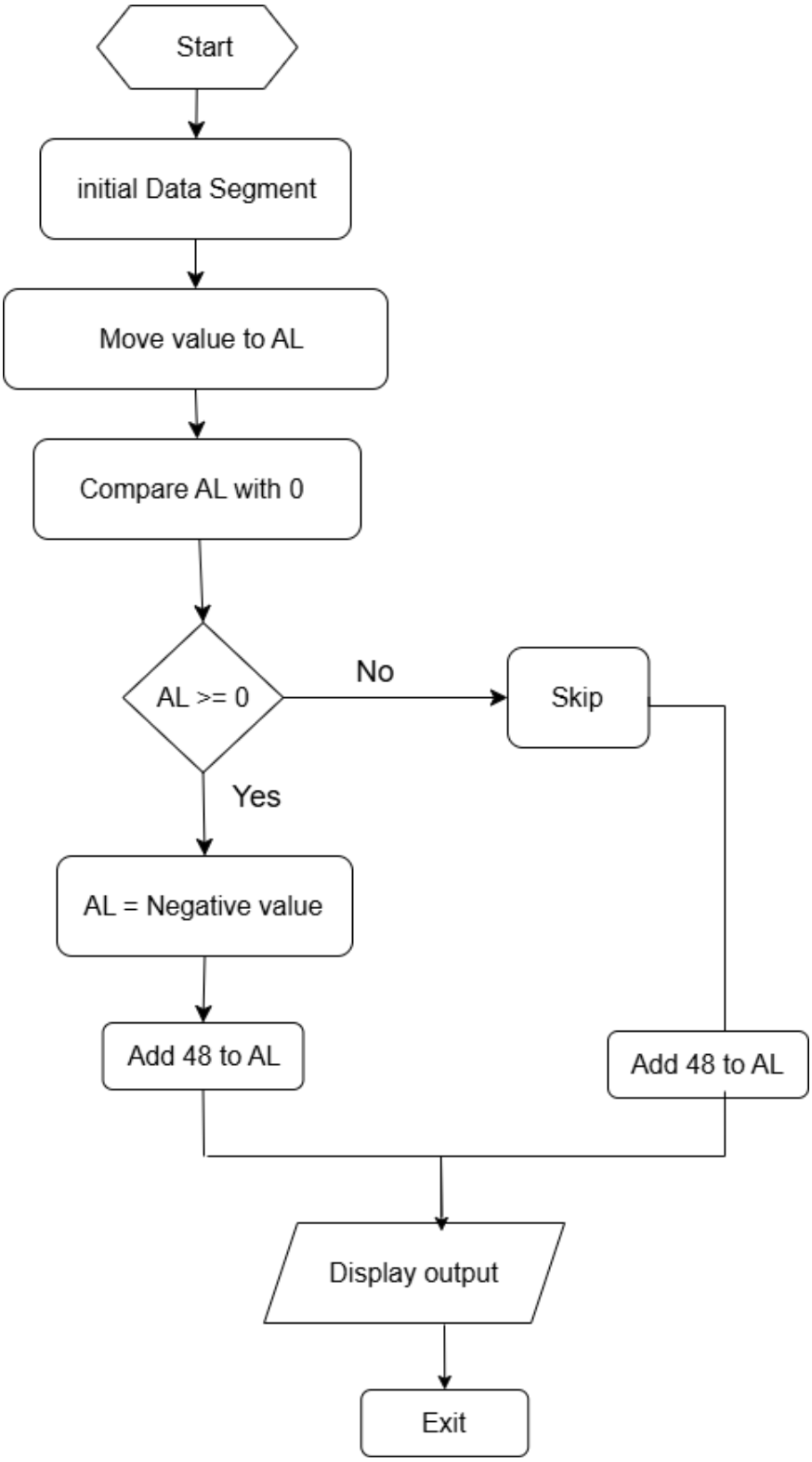
CSE -26

Submitted To:

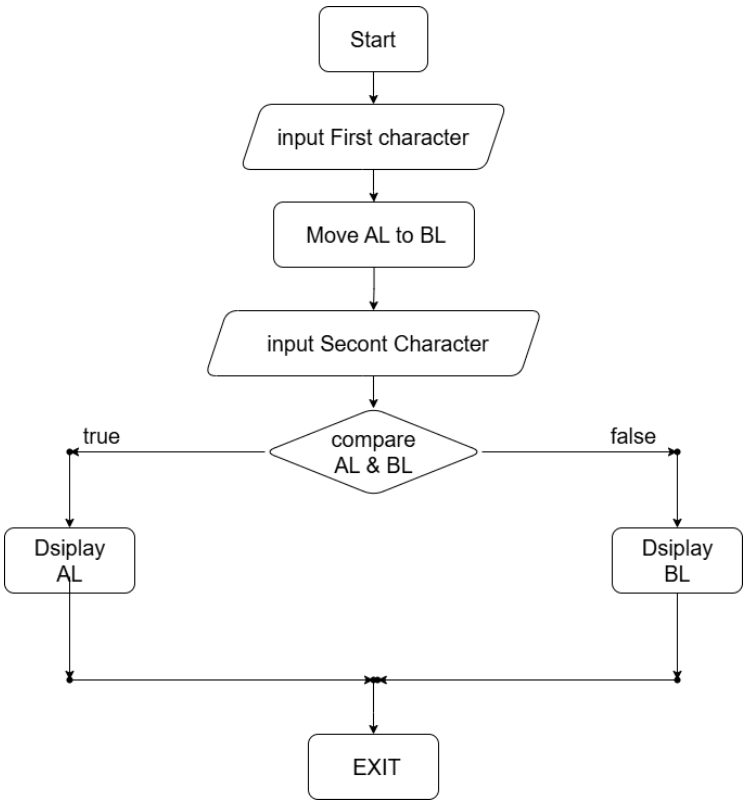
SAIFUR RAHMAN

LECTURER, CSE

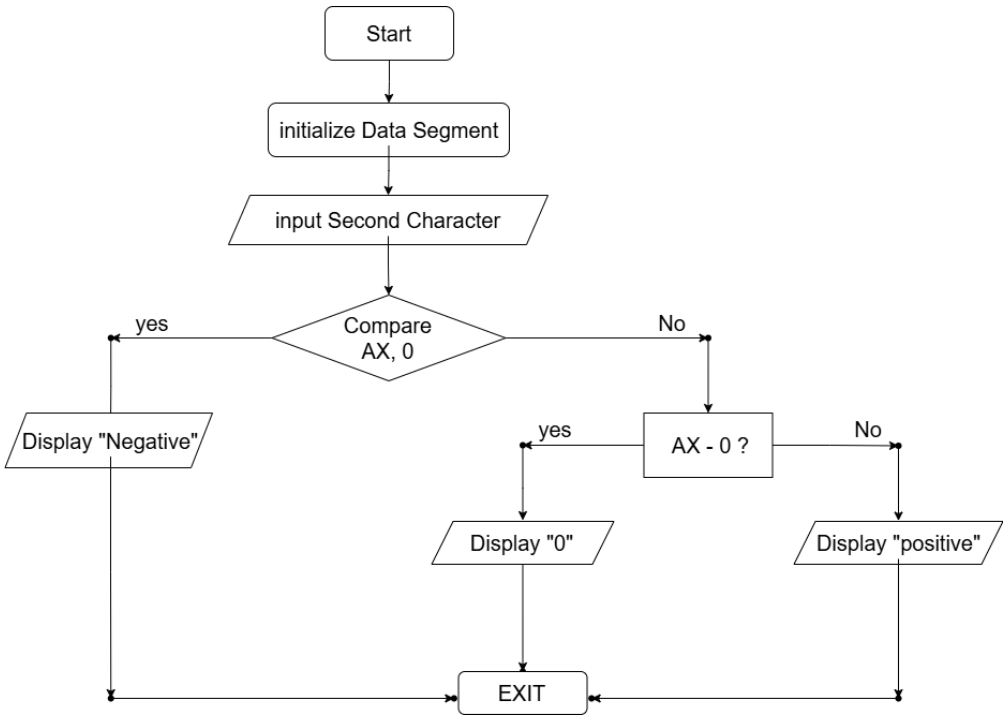
Replace the number in AX by its absolute value:



Suppose AL and BL contain extended ASCII characters. Display the one that comes first in the character sequence:



If AX contains a negative number, put -1 In BX; if AX contains 0, put 0 In BX; if AX contains a positive number, put 1 In BX:

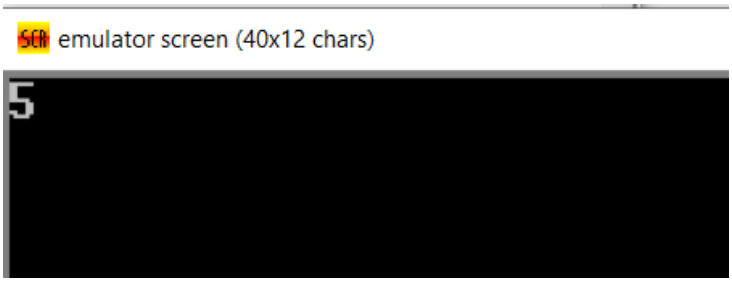


Code:

Replace the number in AX by its absolute value:

```
.model small
.stack 100h

.data

.code
main proc
    ; Initialize the data segment
    mov ax, @data
    mov ds, ax

    ; Move -5 into AL (input value)
    mov al, -5

    ; Check if AL is less than 0
    cmp al, 0
    jge nothing ; Jump to 'nothing' if AL is greater or equal to 0
    neg al      ; Otherwise, negate AL

nothing:
    ; Convert number to ASCII
    add al, 48

    ; Prepare for character output
    mov ah, 2
    mov dl, al

    ; Output the character
    int 21h

    ; Terminate the program
    mov ah, 4ch
    int 21h

exit:
    mov ah, 4ch
    int 21h
main endp

end main
```

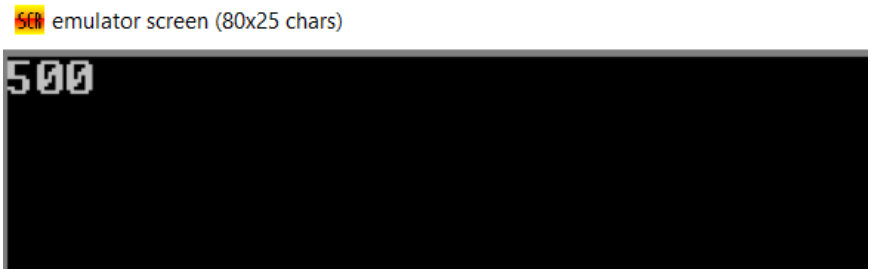
Output:



Suppose AL and BL contain extended ASCII characters. Display the one that comes first in the character sequence:

```
.model small
.stack 100h

.data

.code
main proc
    ; First input character
    mov ah, 1           ; Single character input
    int 21h
    mov bl, al

    ; Second input character
    mov ah, 1           ; Single character input
    int 21h

    ; Check if al < bl
    cmp al, bl
    jl display_al

    ; Else,
    mov ah, 2           ; Single character output
    mov dl, bl
    int 21h             ; Display bl
    jmp exit

display_al:
    mov ah, 2           ; Single character output
    mov dl, al
    int 21h             ; Display al

exit:
    mov ah, 4ch         ; Exit program
    int 21h

main endp
end main
```

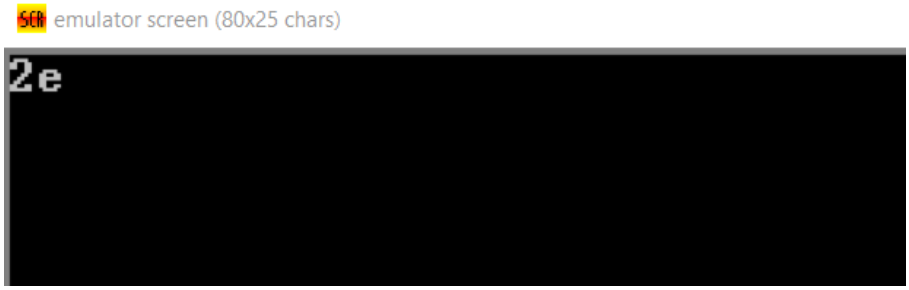
Output:



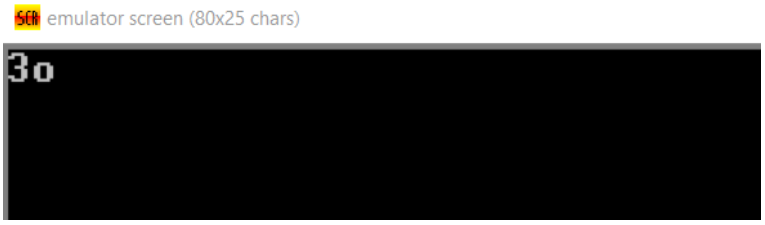
If AL contains 1 or 3, display "o"; if AL contains 2 or 4, display "e":

```
.model small
.stack 100h

.data

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ah, 1            ; Single character input
    int 21h

    ; Check odd
    cmp al, '1'
    je odd
    cmp al, '3'
    je odd

    ; Check even
    cmp al, '2'
    je even
    cmp al, '4'
    je even

odd:
    mov dl, 'o'
    jmp display        ; Skip the even label

even:
    mov dl, 'e'

display:
    mov ah, 2           ; Single character output
    int 21h             ; Show output

    mov ah, 4ch         ; Exit program
    int 21h

main endp
end main
```

Output-Even:



Output-Odd:

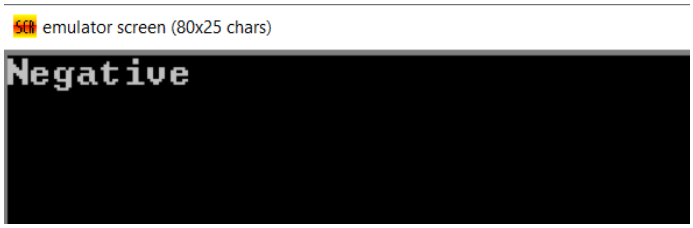


If AX contains a negative number, put -1 In BX; if AX contains 0, put 0 In BX; if AX contains a positive number, put 1 In BX:

```
.model small
.stack 100h

.data
Message_neg db 'Negative', 0dh, 0ah, '$'
Message_zero db 'Zero', 0dh, 0ah, '$'
Message_pos db 'Positive', 0dh, 0ah, '$'

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, -5          ; -5 input
    cmp ax, 0
    jl neg
    je zero
    jg pos

neg:
    mov bx, -1
    mov ah, 9
    lea dx, Message_neg
    int 21h
    jmp exit

zero:
    mov bx, 0
    mov ah, 9
    lea dx, Message_zero
    int 21h
    jmp exit

pos:
    mov bx, 1
    mov ah, 9
    lea dx, Message_pos
    int 21h
    jmp exit
```

```
exit:
    ; Exit program
    mov ah, 4ch
    int 21h

main endp
end main
```

Output:



Conclusion: The assembly code examples illustrate fundamental concepts like setting up data and code segments, handling input/output, conditional logic, and array manipulation. These basics provide a strong foundation for understanding low-level programming and efficient hardware control.